

SMART LENDER: AI-Powered Loan Eligibility Prediction

Project Description

SMART Lender is a web-based, end-to-end artificial intelligence application designed to automate and simplify credit risk assessments in the retail banking sector. By replacing slow, subjective manual underwriting processes with a tuned **XGBoost machine learning classifier**, the application analyzes applicant profiles (including income, dependents, education, and credit history) to determine loan approval eligibility in under 200 milliseconds.

The system features a custom-designed, responsive, warm champagne-gold and cream UI dashboard, an interactive 3-step application form wizard, and a local SQLite database powered by Flask-SQLAlchemy to audit, track, and manage prediction history logs.

Problem Statement

Traditional retail bank loan evaluations rely heavily on manual verification of physical credit files. This approach suffers from several key bottlenecks:

- * **Operational Sluggishness:** Processing a single application manually takes days, leading to low customer satisfaction and backlogs.
- * **Decision Inconsistency:** Manual underwriting is highly subjective and vulnerable to individual cognitive fatigue, leading to decision bias.
- * **Inaccurate Risk Assessments:** Basic rule-based models fail to catch complex, non-linear relationships among variables, resulting in higher loan defaults and bad debt.

Project Objectives

- * **Automate Assessments:** Reduce evaluation times from days to milliseconds.
- * **Minimize Credit Risks:** Optimize classification metrics using a hyperparameter-tuned machine learning engine to prevent defaults.
- * **Provide Auditing Tools:** Create an integrated relational database logs screen to track, review, and purge historical runs.
- * **Expose a Modern Interface:** Develop an intuitive, mobile-ready web portal with interactive data visualization widgets.

Project Scope

The project covers the entire machine learning and software engineering lifecycle:

- * **Data Collection & Cleaning:** Reading the training dataset, replacing null numeric values with their column **mean**, and null categorical values with their **mode**.
- * **EDA Analysis:** Visualizing dataset distributions, gender proportions, and multivariate income vs approval correlations.
- * **Data Preprocessing:** Label encoding text fields, implementing **SMOTE** (Synthetic Minority Over-sampling Technique) to balance class ratios, and normalization via `StandardScaler`.
- * **Model Comparison:** Implementing and evaluating Decision Tree, Random Forest, KNN, Gradient Boosting, and XGBoost classifiers.
- * **Deployment & Scaling:** Running a Flask web server, mapping multi-step UI form inputs to numeric metrics, and hosting the port on a secure public staging URL using Pinggy.

Scenarios

Scenario A: High-Eligibility Approval (Successful Loan Application)

Applicant Profile: The applicant is graduate and married, with an individual monthly income of \$7,500 and a co-applicant income of \$2,500. They are applying for a loan amount of \$180,000 and possess a good credit history (credit score code 1.0).

System Action: The user inputs details into the 3-step application wizard. The frontend performs client-side field validation and posts the form data to the Flask backend router (app.py). The application extracts variables, maps categorical features to their numeric equivalents, and processes continuous parameters using a fitted StandardScaler (scale1.pkl). The normalized input is fed into the tuned XGBoost model binary (rdf.pkl).

Outcome: The XGBoost model calculates class probabilities and returns a positive prediction value (eligibility approval) with a confidence level exceeding 85%. The application UI renders a green success confirmation page stating "Loan will be Approved" and stores the data record into the local SQLite database (smart_lender.db) with an auto-assigned application number.

Scenario B: Poor Credit History Rejection (High Risk)

Applicant Profile: The applicant is graduate and single, with a strong individual monthly income of \$9,000. They are applying for a loan of \$100,000, but have a bad credit history (credit score code 0.0, indicating a past default).

System Action: The form data is posted to /predict. Features are parsed and scaled. While the applicant has high income and a moderate loan amount request, the preprocessed vector indicates a prior credit default risk.

Outcome: The XGBoost classifier prioritizes the credit history parameter and predicts a negative outcome (class 0). The interface displays a warning screen stating "Loan will Not be Approved," highlighting high credit risk. The application entry is logged into the SQLite database history with a rejection status.

Scenario C: Insufficient Income Rejection (High Debt-to-Income Ratio)

Applicant Profile: The applicant is non-graduate and married, with a low individual monthly income of \$1,800 and no co-applicant income (\$0). They are requesting a high loan amount of \$350,000, but have a good credit history (1.0).

System Action: The application inputs are processed and scaled. The system evaluates the

disproportionately high debt-to-income ratio (asking for a \$350k loan with only \$1.8k monthly income).

Outcome: Despite the positive credit history, the model flags the financial imbalance as a major default threat and outputs a negative prediction (class 0). The UI renders the "Loan will Not be Approved" status page, and the application is stored in the database logs.

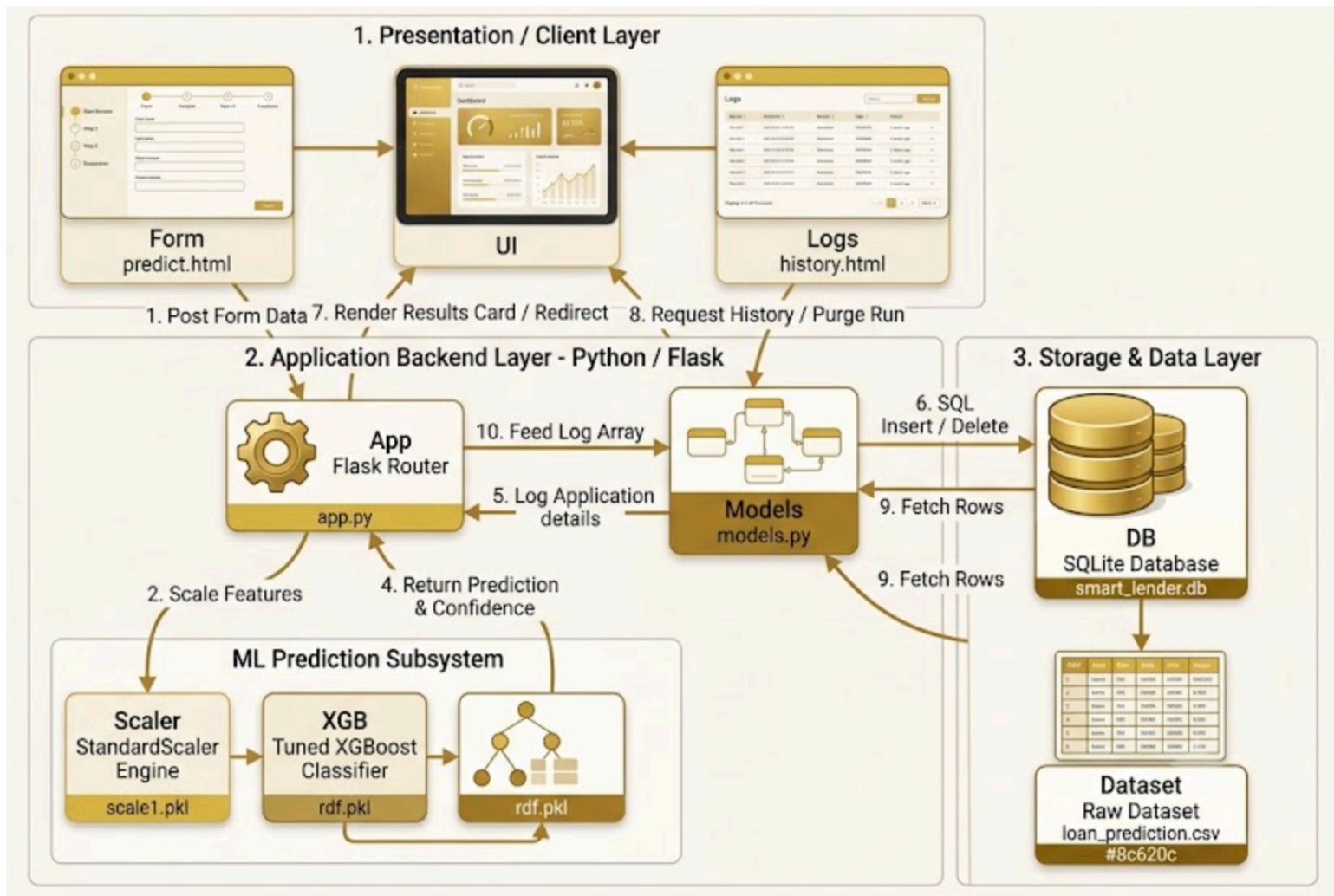
Scenario D: Administrator Log Management (Purging History)

User: A Senior Credit Officer or System Administrator.

System Action: The administrator accesses the prediction logs page at /history to review historical records. They identify test inputs or obsolete records and click the "Delete" button corresponding to a specific entry ID. The Flask backend triggers a database query to purge that transaction row.

Outcome: The application executes a clean deletion command from the SQLite database. The log interface refreshes dynamically, showing the updated dataset without the deleted entry, maintaining data integrity.

Technical Architecture:



Technical Architecture Description

1. Presentation Layer (Client Side)

This is the user-facing tier of the application, designed with a premium, responsive CSS layout styling:

Home Dashboard (home.html): Renders visual Exploratory Data Analysis (EDA) charts (such as loan approval status distributions and model accuracy charts) to give users immediate insights.

3-Step Prediction Wizard (predict.html): Guides the user through a multi-step questionnaire (Personal Details → Financial Info → Loan Requirements) with live validation to capture clean applicant data.

Prediction Logs Screen (history.html): Displays past predictions from the database, allowing users to

audit individual submissions or request record deletions.

2. Application Layer (Backend Logic)

The server execution layer that acts as the controller and orchestrator of the project:

Flask Web Server (app.py): Handles URL routing, form submissions, and database operations.

ML Prediction Engine:

Feature Transformation: Text inputs (like Education, Marital Status) are encoded into numeric values.

Scaling Engine (scale1.pkl): Applies the pre-trained StandardScaler transformations to ensure uniform numeric variance for features like income and loan amount.

XGBoost Classifier (rdf.pkl): Loads the hyperparameter-tuned model to run predictions in under 200 milliseconds, outputting a probability confidence score.

Database ORM (models.py): Utilizes Flask-SQLAlchemy to map Python classes directly to database tables, handling record insertions, lookups, and deletions.

3. Storage Layer (Data Persistence)

This layer manages state persistence and serializations:

SQLite Database (smart_lender.db): A lightweight relational database containing tables for user inputs, evaluation scores, and log history.

Pickle Files (.pkl): Binaries storing the trained model parameters, scaler coefficients, and classification

performance statistics.

Step-by-Step Data Flow

Request: The user submits the loan application form on the frontend UI.

Process & Transform: The Flask app parses the form, runs numeric transformations, and scales values using `scale1.pkl`.

Classify: The normalized vector is evaluated by the serialized XGBoost model (`rdf.pkl`), returning whether the loan should be approved or rejected.

Log: The server inserts the applicant details along with the prediction result into the SQLite database.

Respond: The backend renders the outcome page (`submit.html`) with the prediction state (Approved/Not Approved) and confidence percentage.

Pre-requisites:

- Python Programming
- Machine Learning Fundamentals
- Flask Framework
- HTML
- CSS
- JavaScript
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- Git & GitHub
- Render Deployment

Project Workflow

Milestone 1: Project Conception & Local Environment Setup

Define the Project Scope: Specify the application goals: an AI-driven web application to predict loan approval eligibility based on applicant criteria (income, credit history, marital status, education, etc.) using multiple machine learning models (with tuned XGBoost as the primary model), saving history records, and displaying performance metrics.

Initialize Workspace: Create the local repository directory structure:

/Dataset/ to store raw training files.

/static/css/ and /static/images/ for stylesheets and visual plots.

/templates/ to store layout-base and sub-page HTML files.

Configure Python Virtual Environment: Set up an isolated python virtual environment (python -m venv venv) locally and activate it.

Acquire Training Dataset: Source the target CSV dataset (loan_prediction.csv), which contains historical records of loan applicants including characteristics and final loan approval outcomes (Loan_Status). Move it into the /Dataset/ directory.

Install Core Local Packages: Install standard packages locally: pandas, numpy, scikit-learn, xgboost, imbalanced-learn, matplotlib, seaborn, flask, flask-sqlalchemy, and locust.

Freeze Dependencies: Generate the local requirements.txt file (pip freeze > requirements.txt) to record all working dependency versions.

Milestone 2: Exploratory Data Analysis (EDA) & Machine Learning Pipeline

Data Loading: Read loan_prediction.csv into a Pandas DataFrame. Standardize column names and inspect missing value counts per feature.

Univariate Analysis: Generate distribution charts (using Matplotlib and Seaborn) for continuous variables like ApplicantIncome to examine skewness, and count plots for categorical elements like Gender, Education, and Married. Save plots to /static/images/plots/.

Bivariate Analysis: Plot correlations between categorical inputs and the target Loan_Status (e.g., Credit History vs Loan Status, Education vs Loan Status).

Data Cleaning & Mappings:

Map text categorical features to numeric equivalents (e.g., Gender: Male -> 0, Female -> 1, Married: No -> 0, Yes -> 1).

Perform missing value imputation (e.g., filling missing numerical variables using the median, and categorical variables using the mode).

Perform type casting to ensure all features are represented as clean integer/float arrays.

Multivariate Swarm Plotting: Generate a multivariate swarm plot comparing applicant income, gender, and loan approval status to visualize multi-dimensional feature behaviors.

Apply SMOTE (Synthetic Minority Over-sampling Technique): Balance the target variable (Loan_Status) classes to eliminate biases toward approved loans by generating synthetic records for the minority class.

Feature Scaling: Apply StandardScaler to normalize numeric ranges, ensuring features like income (scale of thousands) do not dominate binary indicators.

Train Baseline Classifiers: Split the dataset into training (67%) and testing (33%) partitions. Train baseline models:

Decision Tree

Random Forest

K-Nearest Neighbors (KNN)

Gradient Boosting

Tune XGBoost Model: Perform hyperparameter tuning on the XGBoost classifier using RandomizedSearchCV with 5-fold cross-validation. Optimize parameters such as learning rate, tree depth, estimators, and regularizations.

Evaluate Performance: Compare model metrics (accuracy, precision, recall, F1-score). Save confusion matrix heatmaps and comparison bar charts.

Export Machine Learning Binaries: Serialize the scaler object (scale1.pkl), testing metrics (metrics.pkl), and the full model bundle (rdf.pkl containing model, scaler, feature list, and metrics) using the Python pickle library.

Milestone 3: Database Design & Core Application Backend

Configure ORM (SQLAlchemy): Initialize SQL Alchemy inside the Flask backend and set up the connection string for a local SQLite database (smart_lender.db).

Define Database Models: Define relational database models in models.py:

User: Stores administrative/user credentials.

ApplicantProfile: Stores mapped applicant parameters (e.g., Gender, Married, Education).

CreditHistory: Houses credit score estimates.

LoanApplication: Stores requested loan amounts, income criteria, and terms.

PredictionResult: Logs the model prediction outputs, calculated probabilities, inference times, and model names.

ModelMeta: Keeps track of current machine learning validation accuracies.

Database Initialization: Set up startup context triggers to call db.create_all(), automatically creating SQLite tables if

they do not exist.

Model Loading Routine: Implement a model loading routine that loads `rdp.pkl` into global memory (`MODEL_DATA`).

Implement Prediction API Route: Create the `/predict` POST route to:

Parse form inputs.

Commit applicant details to `ApplicantProfile` and `LoanApplication` SQL tables.

Construct the feature vector, scale it using the loaded scaler, run inference via XGBoost, and record predictions inside the `PredictionResult` table.

Milestone 4: Premium UI/UX Design & Frontend Integration

Design System & Styling: Create `style.css` containing variables for a dark-mode theme, sleek gradients, glassmorphism card properties (blur, transparent borders), and transition animations.

Establish Base Layout: Design `layout.html` containing the navbar, responsive structural containers, footer, and font integrations (Google Fonts Outfit/Inter and FontAwesome Icons).

Build the Home Dashboard: Design `home.html` featuring:

Metric indicator cards showing model training accuracy statistics.

Interactive charts (using the generated EDA images) organized into tabs.

Action buttons routing to the predictor wizard.

Develop Multi-step Wizard: Create the multi-step form wizard inside `predict.html` with:

Interactive steps (Step 1: Personal Info, Step 2: Income Details, Step 3: Credit History).

Clean Javascript animations controlling active steps.

Visual form validation indicators.

Design Prediction Results Screen: Design `submit.html` to display prediction results with:

Large status alerts (e.g., Green glass card for "Approved", Red glass card for "Rejected").

Probability score progress bars.

Action buttons to check history or run a new prediction.

Implement History Log: Build `history.html` containing a table displaying past submissions, prediction configurations, confidence levels, and database record deletion actions.

Milestone 5: Local Testing & Validation

Run Local Server: Launch the local development server (`python app.py`) and verify form input routing, model execution latency, and SQLite persistence.

Perform Load Testing: Create a `locustfile.py` and trigger a Locust performance test suite. Run high-concurrency requests to the `/predict` endpoint to verify that model loading does not cause memory leaks or request timeouts.

Debug Edge Cases: Verify form errors (empty values, text inside number fields) are handled cleanly with user-facing flash warning alerts.

Milestone 6: Deploying on PythonAnywhere (Making it Public)

Obtain API Credentials: Retrieve the PythonAnywhere username (KVNVK) and API Token from the account settings tab.

Synchronize Files: Upload the project directory contents via REST API or Git, ensuring `app.py`, `models.py`, `train_models.py`, `requirements.txt`, the `/templates/` directory, the `/static/` directory, and `/Dataset/loan_prediction.csv` are in place. Exclude heavy local `.pkl` files to avoid environment version mismatches.

Configure Web Application: Create the web app instance on PythonAnywhere targeting the domain `kvnvk.pythonanywhere.com` running Python 3.10. Modify the virtual host WSGI configuration script to point to the `/home/KVNVK/smart_lender/` source directory and import `app` as application.

Install Remote Dependencies: Launch a remote Bash console and run `pip install --user imbalanced-learn` (and check for `pandas/xgboost` dependencies) to populate PythonAnywhere's library scope.

Self-Healing Startup Configurations: Integrate site-packages paths inside the PythonAnywhere app initialization script to allow uWSGI workers to recognize user-level package directories.

Compile Model on PythonAnywhere: Run `train_models.py` directly inside the PythonAnywhere host directory. This compiles the XGBoost models using the server's local NumPy (1.x) and SciPy configurations, generating compatible `rdf.pkl` and `scale1.pkl` files to resolve any `numpy._core` loading mismatches.

Perform Health Checks: Run requests to `http://kvnvk.pythonanywhere.com/predict` to ensure the server correctly routes to `result/<id>` and returns predictions.

Milestone 7: Storing & Publishing Code on GitHub

Initialize Git Repository: Run `git init` inside the local project workspace.

Create Gitignore Rules: Write a `.gitignore` file to prevent heavy, temporary, or private files from being tracked:

```
text
venv/
*.pyc
__pycache__/
smart_lender.db
local_logs.log
.DS_Store
```

Commit Code Locally: Stage all workspace changes (`git add .`) and commit them with a descriptive message: `git commit -m "feat: complete Smart Lender application with public deployment config"`

Link to GitHub Repository: Create a new public repository on GitHub named `Smart-Lender`. Copy the remote SSH or HTTPS repository URL and link it locally: `git remote add origin <github-repository-url>`

Push Code to GitHub: Set the default branch to `main` and push your codebase: `git branch -M main git push -u origin main`

Update Repository Documentation: Create a clean `README.md` containing the project overview, tech stack details, installation steps, and the public PythonAnywhere link. Commit and push the final documentation updates.

